

Essay 2: Missing Data Strategies

Missing data is one of those problems that looks trivial on the surface, yet it has the power to reshape a model's behaviour in ways that only become obvious later. Géron (2022) makes it clear that there is no neutral choice here—every missing-data strategy carries an assumption, and those assumptions influence what the model ultimately learns. The core approaches fall into deletion, imputation, prediction-based filling, and hybrids such as adding missingness indicators. The key is understanding when each method makes sense and the kind of distortion it introduces.

1. Removing rows with missing values

The simplest approach is to drop any row containing missing data. This is sometimes acceptable when the proportion of missingness is genuinely small and the dataset is large enough that losing a few rows has no meaningful impact. But this only works well when the values are Missing Completely At Random (MCAR). In reality, that assumption usually breaks down. Rows with missing information often belong to specific subgroups—patients arriving through emergency routes, new customers, high-risk accounts—and removing them systematically erases exactly the cases that carry important behavioural signals. Géron highlights this as the risk: the dataset becomes clean, but also narrower and less representative.

2. Removing columns with missing values

Dropping a feature entirely is appropriate when the missingness is extremely high (e.g., 70–90%) and the feature adds little even when present. Géron notes that some columns with heavy missingness become noise generators, particularly for tree-based models that will happily overfit their irregular patterns. But dropping a column is a heavier decision than dropping rows because it removes the feature's signal entirely. If we drop the satisfaction column, for example, we lose a valuable target variable (although this depends on what the target is), and we also remove any possibility of the model learning how satisfaction interacts with arrival patterns or service types. Sometimes dropping the feature is the right trade-off, but it needs to be justified, not automatic.

3. Traditional imputation (mean, median, mode)

Imputation is often the most balanced first step. Scikit-learn's SimpleImputer, discussed extensively in Géron's preprocessing chapters, was designed for this exact purpose. Mean imputation for continuous variables, median for skewed features, and mode for categoricals preserve the data structure without discarding rows. The drawback is subtle: these strategies compress the natural spread of the data. Géron points out that replacing a range of values with a single central statistic reduces variance and can make models appear more confident than they should be. Linear models lose nuance; distance-based models misinterpret artificially similar points; and overall uncertainty is understated.

4. Imputation combined with a missingness indicator

One practical enhancement is pairing imputation with an explicit binary flag indicating whether the value was originally missing. This preserves the row count while also acknowledging that the absence of data can itself be meaningful. In many datasets, missingness carries a behavioural or operational signal—patients without satisfaction scores may cluster in ICU; customers with missing KYC fields may belong to specific financial segments. Géron treats this approach as a useful middle ground: the model gets a complete feature matrix, but it also sees where the data was missing rather than having that information silently erased.

5. Prediction-based imputation (k-NN, iterative models, etc.)

Prediction-based imputation—using KNNImputer, IterativeImputer, or even separate ML models—tries to reconstruct missing values using relationships across the dataset. Géron presents this as the most expressive technique because it tries to restore the underlying structure rather than flattening it. However, it carries a larger leakage risk. If the imputation is not wrapped inside a scikit-learn Pipeline, the imputer can accidentally “peek” at the test set during fitting. This inflates performance and produces models that look robust but crumble when deployed. When done properly, prediction-based imputation often produces more realistic values and reduces information loss.

Example where the choice changes the model entirely

Consider a Nigerian hospital dataset where 15–20% of satisfaction scores are missing for ICU patients because surveys are harder to administer during critical care. If we drop all rows with missing satisfaction, ICU patients almost disappear, and the model learns an unrealistically optimistic

satisfaction profile. If we drop the satisfaction column entirely, we lose an important target variable and flatten the clinical story. If we apply mean imputation, we erase meaningful variability between ICU and general-medicine patients. But if we impute the values and include a missingness indicator, the model can learn both the likely satisfaction score and the fact that ICU cases systematically produce missing responses. The choice of strategy does not just influence one feature—it changes the entire narrative the model learns.

References:

Géron, A. (2022). Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow. 3rd ed. O'Reilly Media.